Practical - 6

Aim:

To Implement of chain matrix multiplication using dynamic programming.

Theory:

Matrix Chain Multiplication (MCM) using dynamic programming finds the most efficient way to multiply a sequence of matrices by minimizing the total number of scalar multiplications. Because matrix multiplication is associative, the order of parenthesization changes the computational cost drastically, though the final matrix remains identical.

Algorithm:

Suppose there are n matrices:

$A1 \times A2 \times A3 \times ... \times An$

where matrix Ai has dimensions:

$p[i-1] \times p[i]$

1. Read the number of matrices n.
2. Store the dimensions of the matrices in an array p.
3. Create a DP table m[n+1][n+1].
4. Set m[i][i] = 0 because multiplying one matrix requires no multiplication.
5. Consider chain lengths from 2 to n.
6. For every possible starting position i:
    - Calculate j = i + chainLength - 1.
    - Initialize m[i][j] to a very large value.

7. Try every possible split position k between i and j.
8. Calculate the multiplication cost:

m[i][k] + m[k+1][j] + p[i-1] × p[k] × p[j]

9.Store the minimum cost in m[i][j].

10.The final answer is m[1][n]

Program Code

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;
int matrixChainMultiplication(vector<int>& p, int n) {
    // m[i][j] stores the minimum number of
    // scalar multiplications needed to multiply
    // matrices Ai to Aj.
    vector<vector<int>> m(n + 1, vector<int>(n + 1, 0));

    // Chain length
    for (int length = 2; length <= n; length++) {

        for (int i = 1; i <= n - length + 1; i++) {
```

```
        int j = i + length - 1;

          m[i][j] = INT_MAX;

          // Try every possible split
          for (int k = i; k < j; k++) {

             int cost = m[i][k]
                   + m[k + 1][j]
                   + p[i - 1] * p[k] * p[j];

             if (cost < m[i][j]) {
                m[i][j] = cost;
             }
          }
       }
    }

    return m[1][n];
}
```

```
int main() {
    int n;

    cout << "Enter number of matrices: ";
    cin >> n;

    vector<int> p(n + 1);

    cout << "Enter dimensions of matrices:\n";

    cout << "For example, for matrices:\n";
    cout << "A1 = 10x20, A2 = 20x30, A3 = 30x40\n";
    cout << "Enter: 10 20 30 40\n\n";

    for (int i = 0; i <= n; i++) {
        cin >> p[i];
    }

    int minimumCost = matrixChainMultiplication(p, n);
```

```
 cout << "\nMinimum number of scalar multiplications = "
      << minimumCost << endl;
   cout<<"kolusu uday kiran\n";
   cout<<"92460118175\n";
   cout<<"5-EN18\n";
   return 0;
}
```

Result :